Layered & Modular Architecture Patterns

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

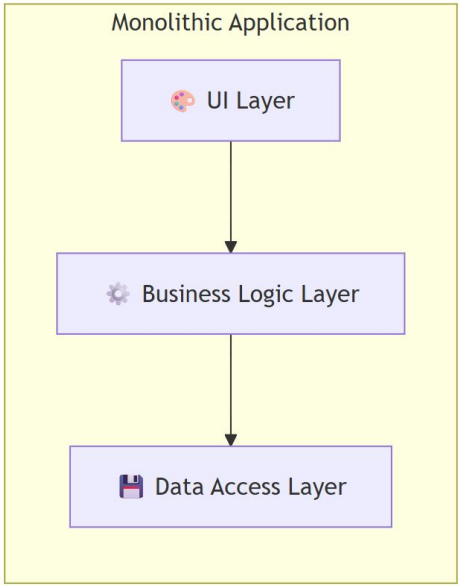
Monolithic and Layered Architectures

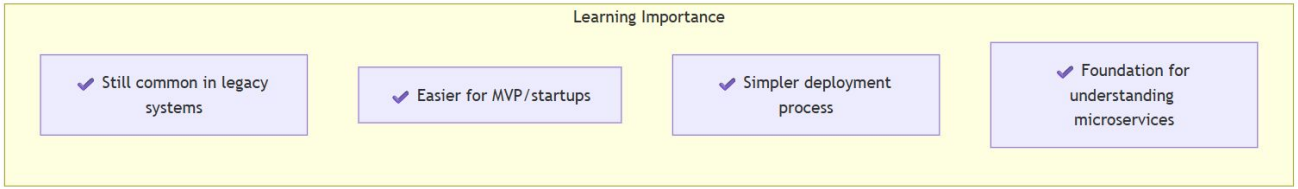
Layered & Modular Architecture Patterns

Mastering Software Architecture Patterns and System Design

Why Study Monolithic Architectures?

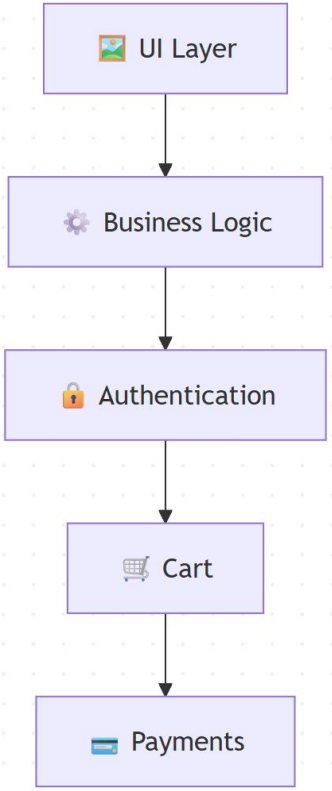
- 🧠 What is a Monolithic Architecture?
  - A monolith is a single, self-contained application where all functionality—UI, business logic, data access—resides in one tightly integrated codebase.
  - Deployed as one unit, runs as one process.
- 📌 Why Learn This First?
  - It's the foundation of most traditional systems.
  - Many companies (especially legacy enterprises) still operate large monoliths.
  - You can’t truly understand microservices or modular architectures without grasping what we’re evolving from.





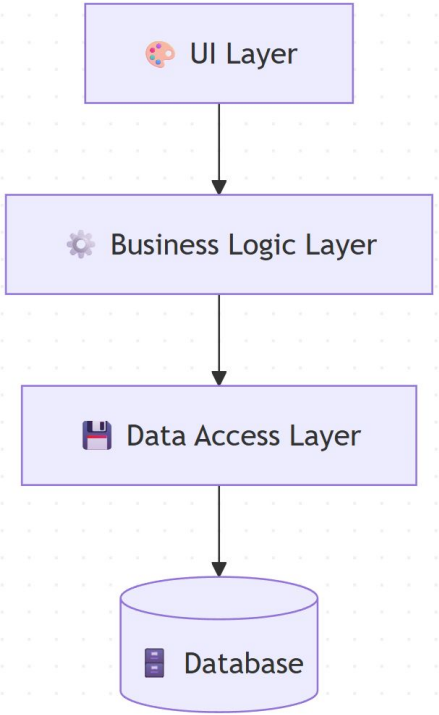
Characteristics of Monolithic Architecture

- Tightly coupled components
- Shared memory and codebase
- Deployed as one unit
- Easy to develop early on
- Hard to scale selectively



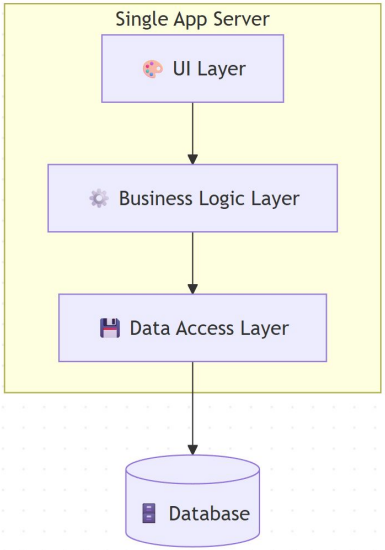
Introduction to Layered Architecture

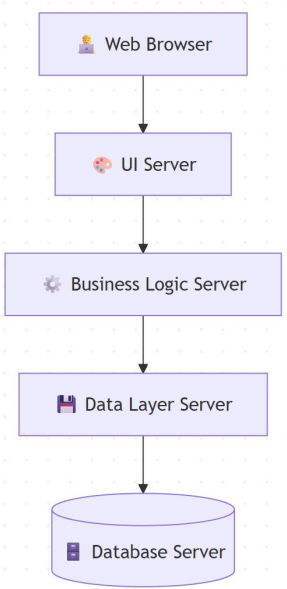
- Layered Architecture = logical separation of concerns
- Typical layers:
  - Presentation/UI
  - Business Logic
  - Data Access
- Often used inside monolithic applications.



3-Tier vs N-Tier Architecture

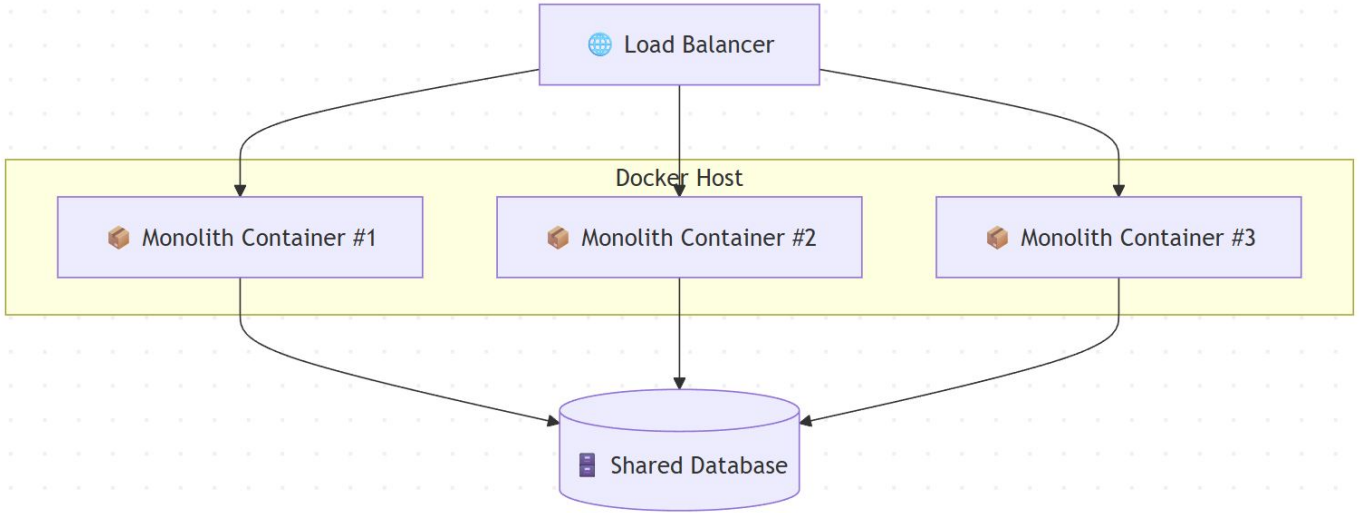
- 3-tier = Presentation, Logic, Data
- N-tier = Same logic but layers may be physically deployed separately
- Adds scalability and flexibility





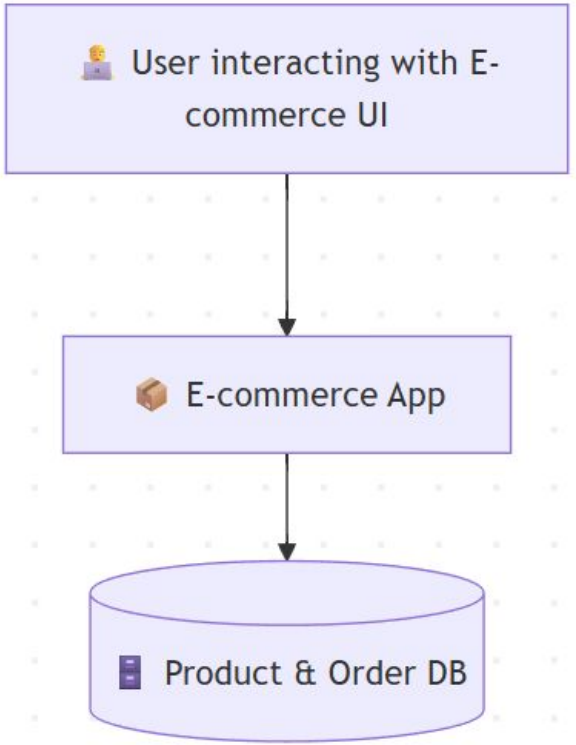
Deployment Model for Monoliths

- Deployed as a single artifact (WAR, EAR, binary, container)
- Scaling = duplicating the whole app (horizontal scaling)
- Small changes require full redeploy



Real-world Example of a Monolith

- Example: A traditional e-commerce website
  - One application handling everything from login to checkout to payment
- Early versions of Amazon, eBay were monoliths



Benefits of Monolithic Architecture

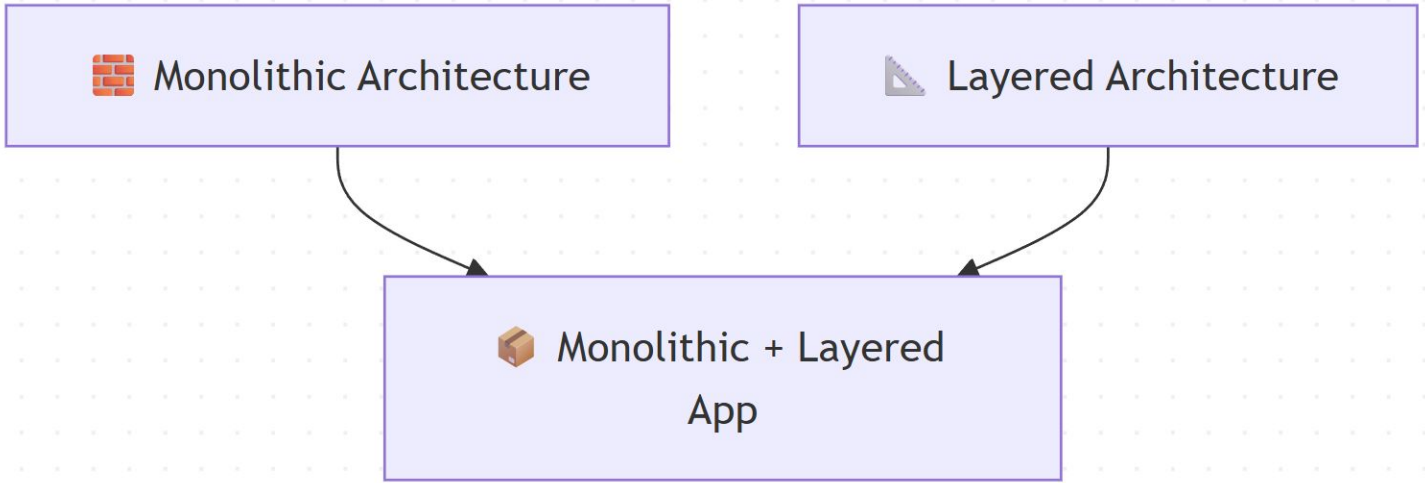
- ✅ Easy to develop initially
- ✅ Simple to deploy
- ✅ Easy testing (single app context)
- ✅ Performance (no network overhead between modules)

Limitations of Monolithic Architecture

- 🚫 Hard to scale specific features
- 🚫 Tight coupling = risk in change
- 🚫 Long build and deployment times
- 🚫 Difficult to adopt new tech stack per module
- 🚫 Barrier to team scaling

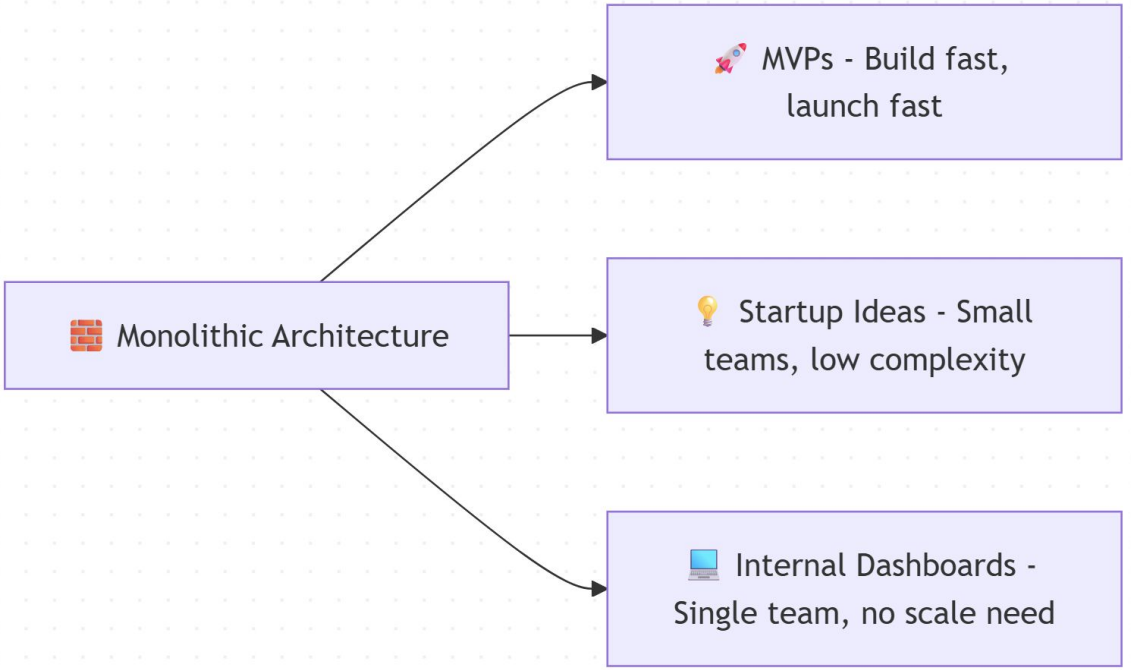
Monolith vs Layered Architecture

- Layered Architecture can exist within monolith
- Monolith = Deployment strategy
- Layered = Design strategy
- Both can coexist



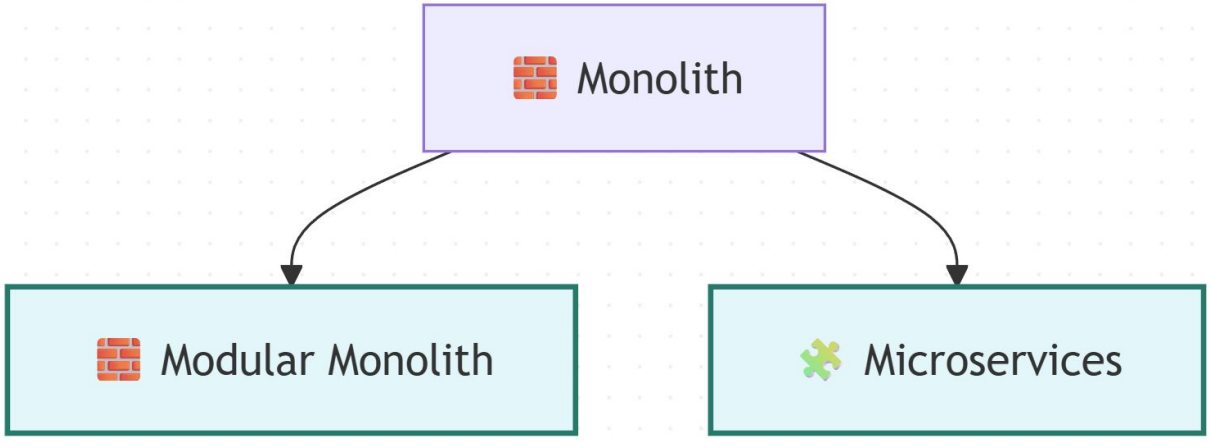
Typical Use Cases for Monoliths

- Startups / MVPs
- Internal tooling
- Single-team projects
- Early-stage products



When to Move Beyond the Monolith

- Product is scaling fast
- Frequent team conflicts on codebase
- Long release cycles
- Scaling and availability issues
- Need for independent deployments



Summary - Monolithic and Layered Architectures

- Monoliths are not bad – just not always scalable
- Layered architecture helps structure large apps
- Understand when to adopt layered monoliths vs distributed systems
- What’s next:
  - Hexagonal Architecture (Ports & Adapters)

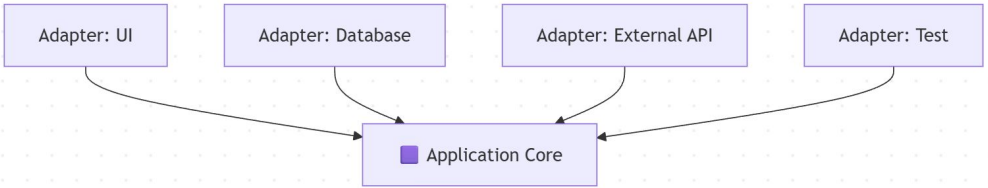
Hexagonal Architecture (Ports & Adapters)

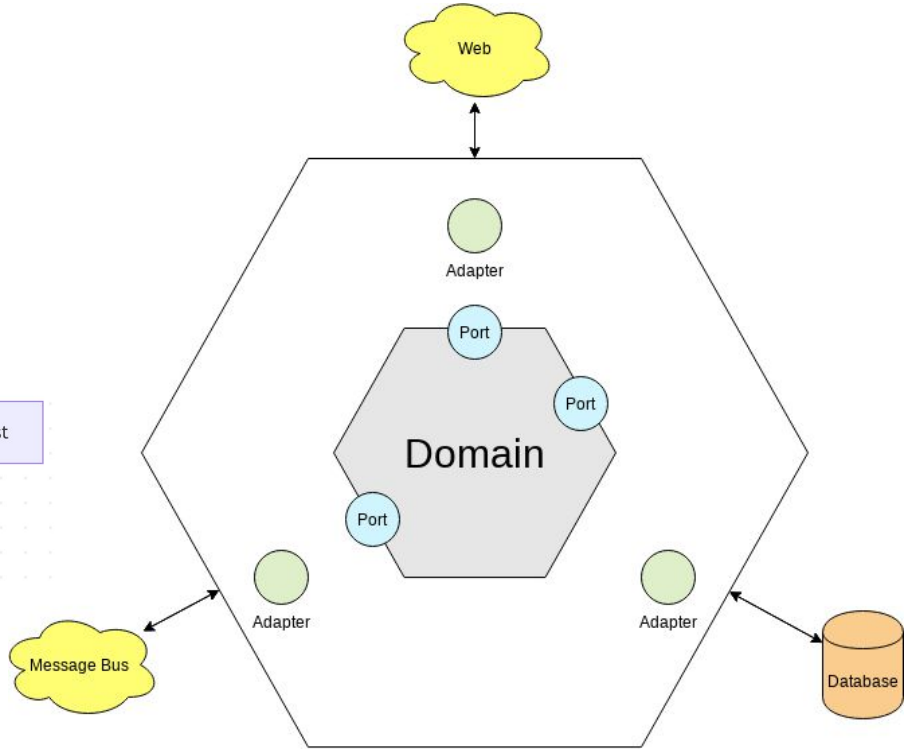
Layered & Modular Architecture Patterns

Mastering Software Architecture Patterns and System Design

What Is Hexagonal Architecture?

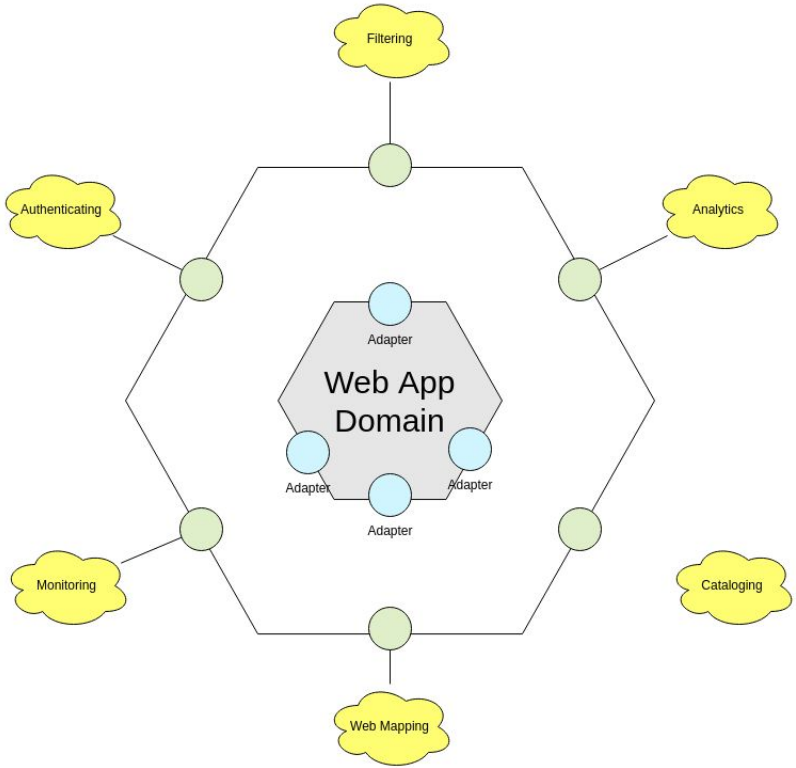
- Core idea: isolate domain logic from external concerns
- Domain logic talks via ports
- External systems implement adapters





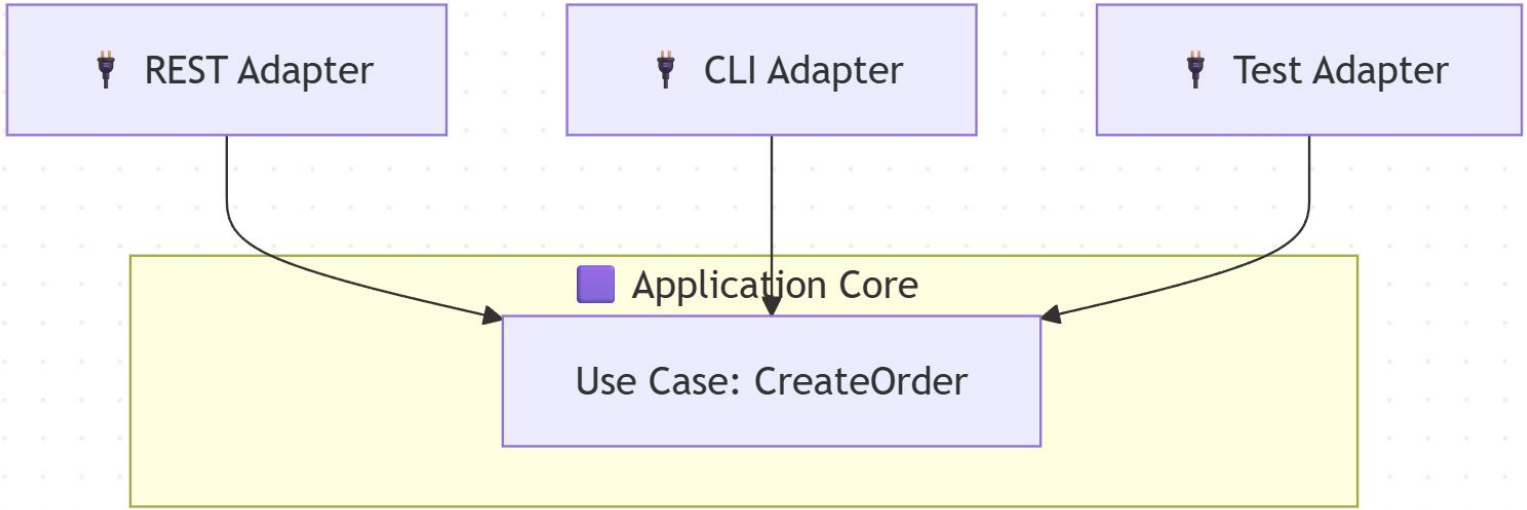
Anatomy of a Hexagonal App

- 🟪 Core
  - The heart of the application — contains domain models, business rules, and use cases.
  - It's pure logic: no frameworks, no I/O code.
- 🔌 Ports
  - Inbound Ports: Define how the outside world can invoke the app (e.g., PlaceOrder, GetInvoice use cases).
  - Outbound Ports: Define what the app needs from external systems (e.g., UserRepository, PaymentService).
- 🔁 Adapters
  - External code that implements the ports — REST controllers, database access, API clients, CLI, tests, etc.
  - Adapters depend on the core, not the other way around.
- 🧭 Direction of Control
  - All flow is inward to the core through well-defined ports.
  - This is what makes Hexagonal Architecture highly testable and modular.



Ports & Adapters = Plug & Play Architecture

- Add/remove adapters without affecting domain logic
- Great for test doubles (mocks, stubs)
- UI or DB can change freely

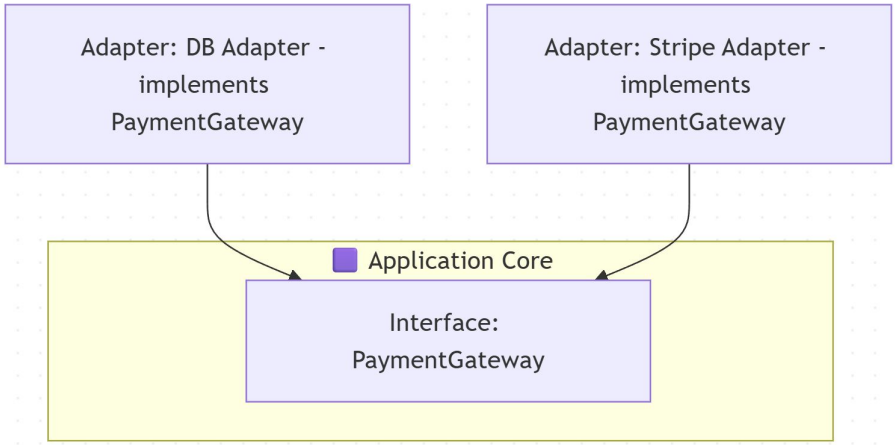


Comparison with Layered Architecture

- 🧱 Layered Architecture
  - Focuses on internal structure (UI → Logic → Data)
  - Common in monolithic apps and early-stage systems
  - Layers often call each other directly and top-down
  - Can become hard to isolate and test individual layers
  - Typically framework-driven (e.g., MVC, Spring, ASP.NET)
- 🟪 Hexagonal Architecture
  - Focuses on I/O boundaries (everything goes through ports)
  - External systems (UI, DB, APIs) communicate via adapters
  - Core is framework-agnostic, just plain business logic
  - Highly testable: easy to use mocks/fakes
  - Promotes clean separation between domain and infrastructure

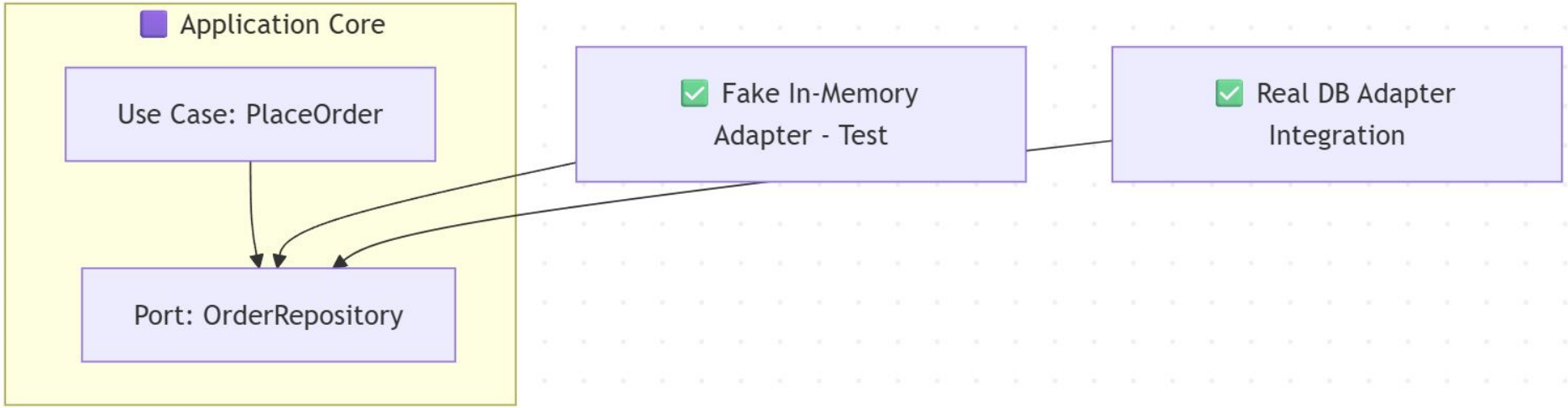
Dependency Inversion in Action

- Core defines what it needs (interfaces)
- Adapters implement how
- Flow is inverted: infrastructure depends on core



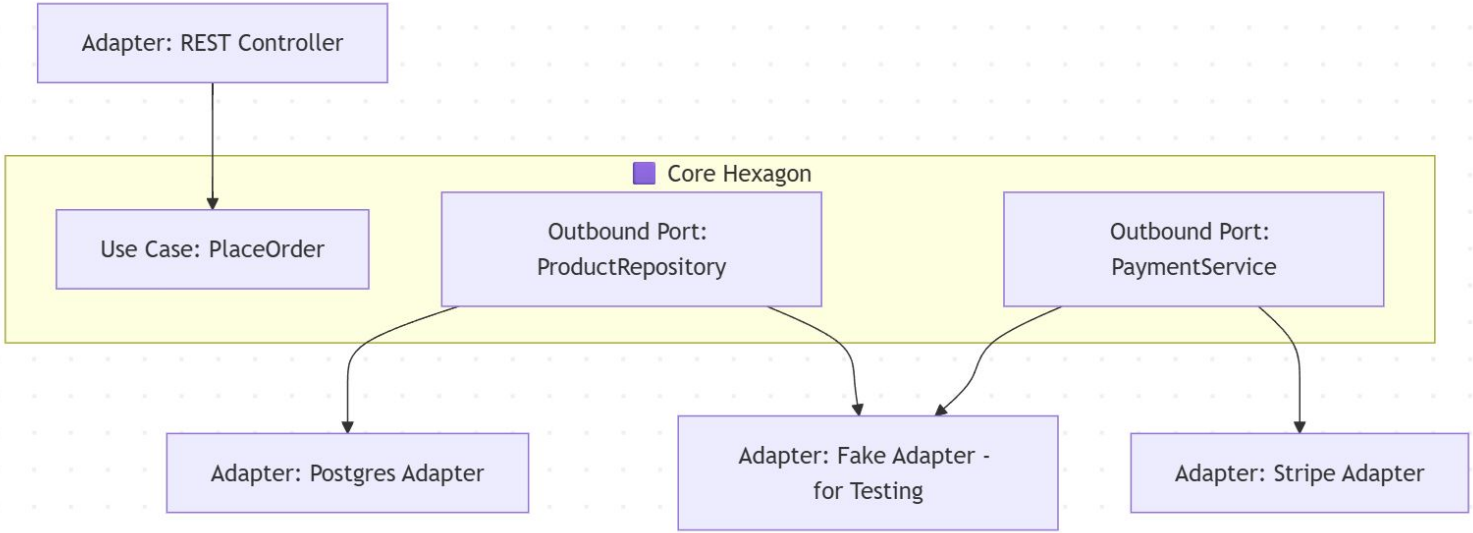
Testing Made Easy

- Fake adapters for tests
- Core logic testable without database or HTTP
- Enables true unit + integration testing



Example Use Case: Place Order

- Use case: PlaceOrder (inbound port)
- Calls ProductRepo and PaymentService (outbound ports)
- Adapters: REST controller, in-memory DB, Stripe client



Summary - Hexagonal Architecture

- Hexagonal Architecture = decoupled, testable, flexible
- Core talks only through ports
- Adapters are interchangeable
- Enables clean separation and better long-term maintainability
- What’s next:
  - Clean vs Onion Architecture

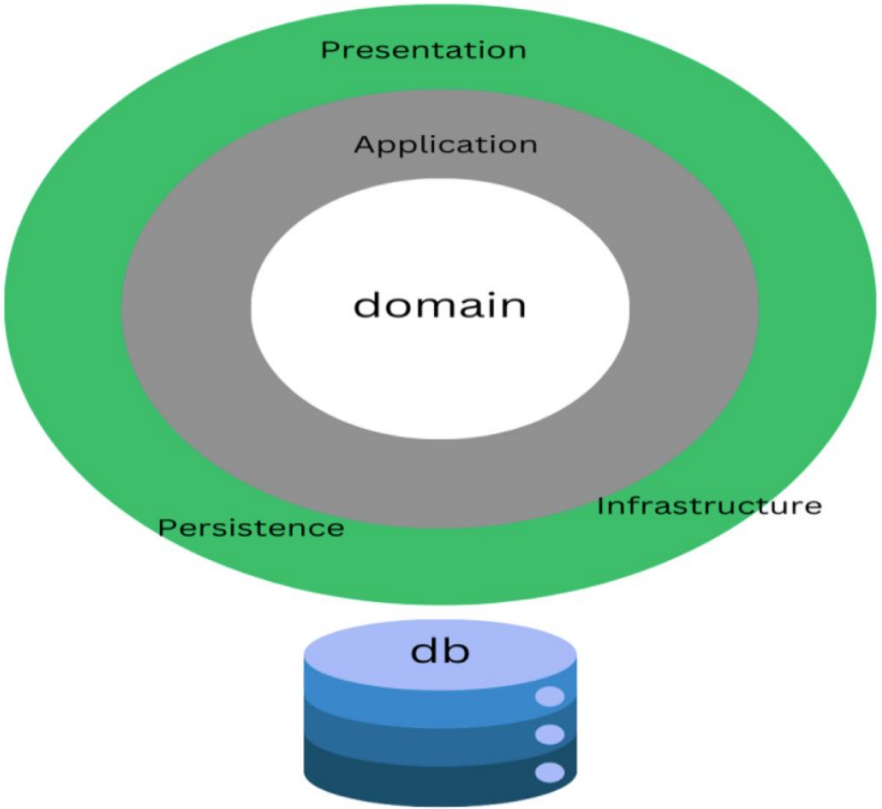
Clean and Onion Architectures

Layered & Modular Architecture Patterns

Mastering Software Architecture Patterns and System Design

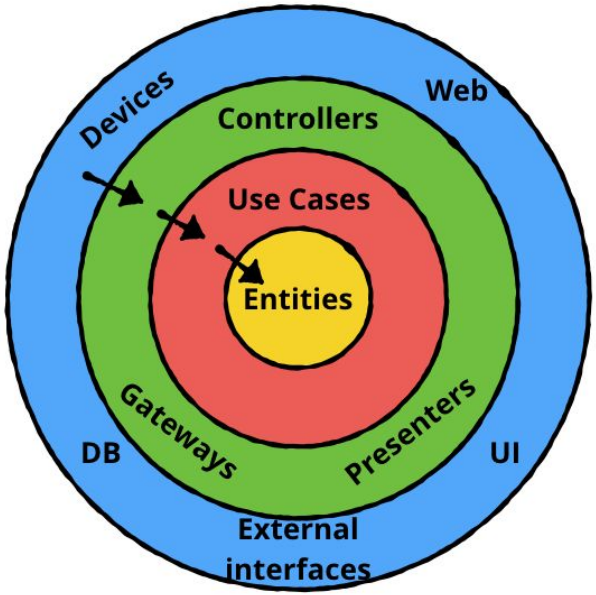
Why Domain-Centric Architecture?

- Keeps business rules independent
- Isolates infrastructure dependencies
- Enables easy unit testing
- Encourages long-term maintainability



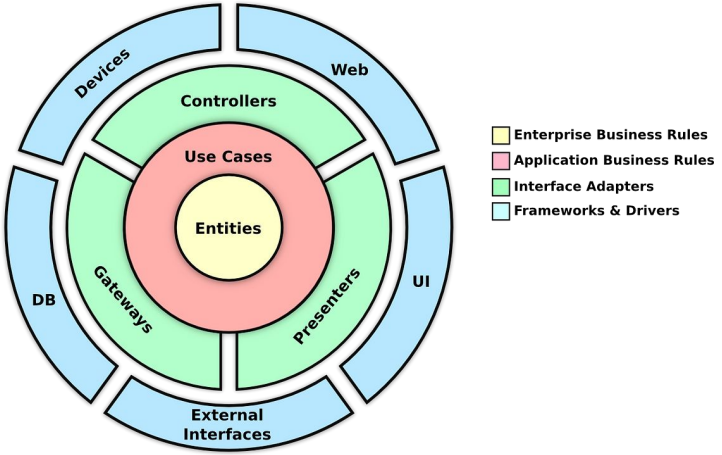
Deep Dive into Clean Architecture

- Invented by Robert C. Martin (Uncle Bob)
- Code organized into concentric circles
- Dependency Rule: Dependencies point inward



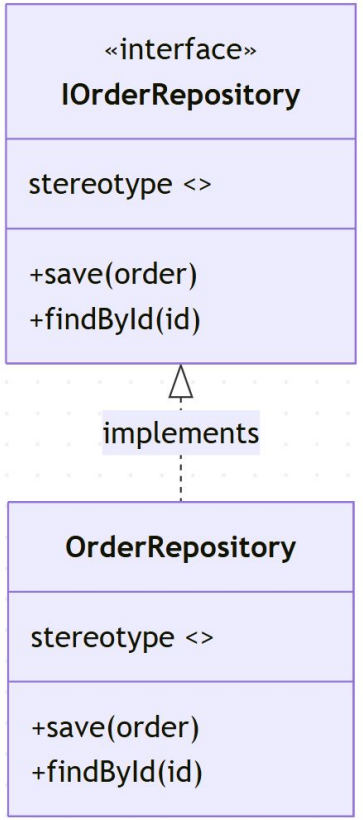
Layers of Clean Architecture

- Entities
  - Contain enterprise-wide business rules
  - Pure business objects (e.g., Order, Customer)
  - Independent of frameworks or infrastructure
- Use Cases (Application Business Rules)
  - Orchestrate interactions between entities
  - Represent specific application behaviors (e.g., PlaceOrder, RegisterUser)
  - Define input/output models and coordinate workflows
- Interface Adapters
  - Convert data from external systems to internal formats
  - Examples: Controllers, Presenters, Gateways
  - Implements interfaces defined in use cases
- Frameworks & Drivers (Infrastructure)
  - Outer layer with UI, DBs, Web APIs, external services
  - Includes tools like ASP.NET, Django, ORMs, messaging platforms
  - Depends on inner layers but not vice versa
- Dependency Rule
  - Code in inner layers should know nothing about outer layers
  - All dependencies point inward toward the core (domain and use cases)



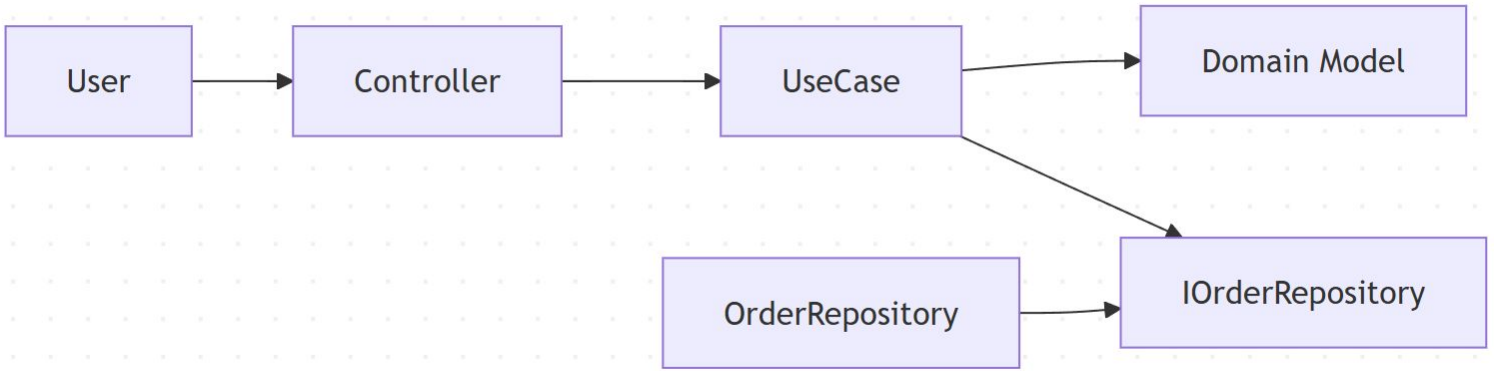
Dependency Inversion in Clean Architecture

- Interfaces defined in inner layers
- Outer layers implement those interfaces



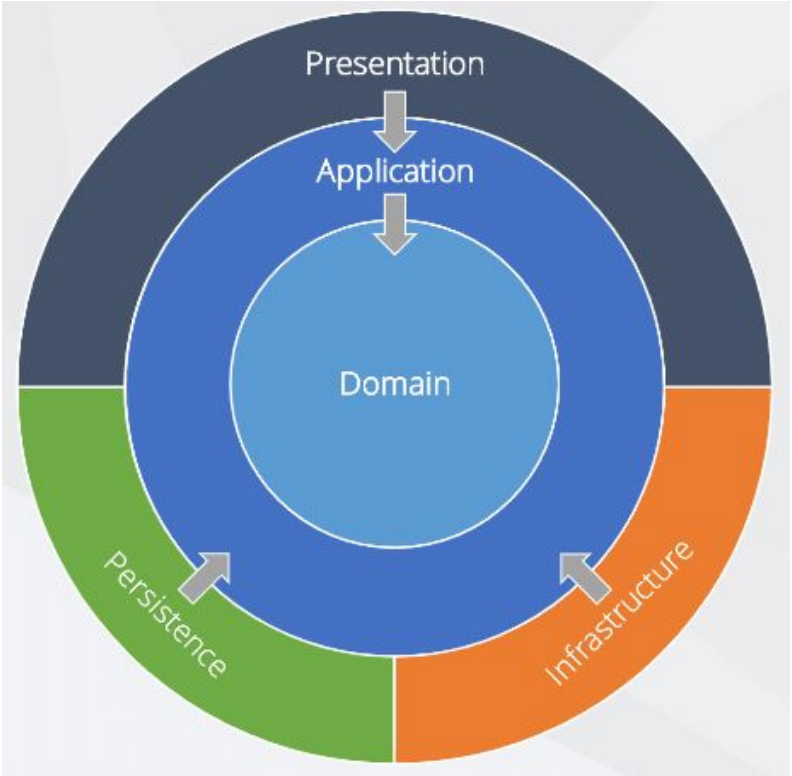
Clean Architecture in Action (Example Use Case)

- Use Case: Place Order
- Flow:
  - Controller receives request
  - Calls Use Case
  - Use Case talks to repository interface
  - Infra implements DB logic



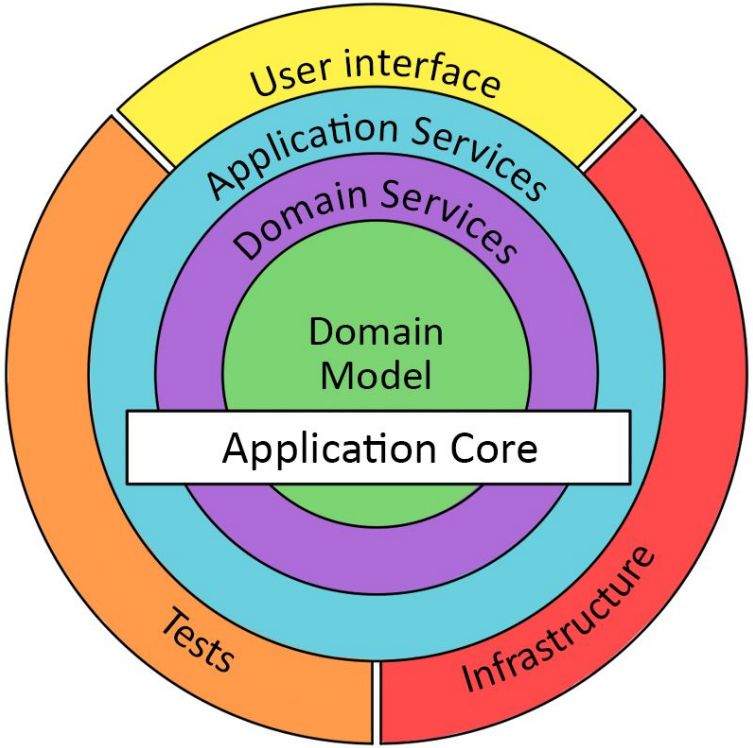
Deep Dive into Onion Architecture

- Proposed by Jeffrey Palermo
- Domain model is at the very core
- Interfaces invert dependencies
- Application services coordinate use



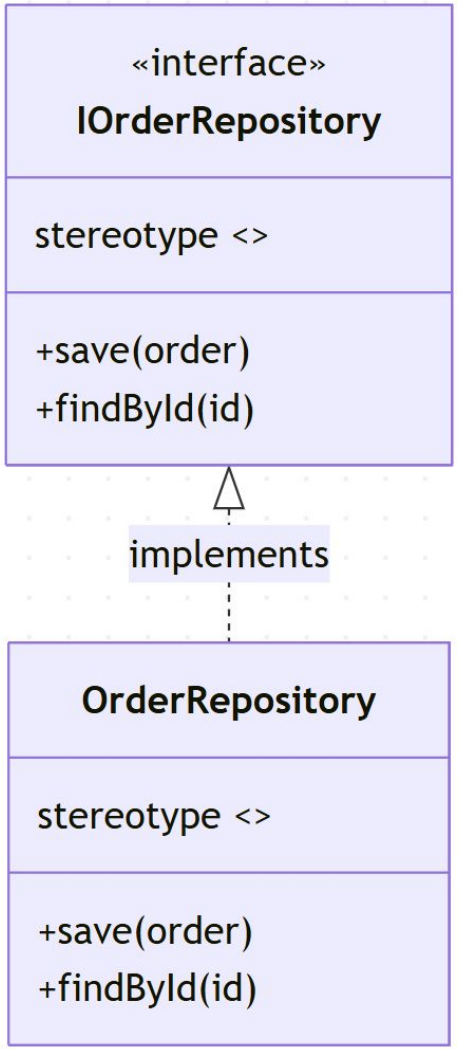
Layers of Onion Architecture

- Domain Model (Core Layer)
  - Contains core business entities and logic
  - Represents aggregates, value objects, and domain rules
  - No dependencies on any other layer
  - Examples: Order, Product, Customer
- Domain Services
  - Encapsulate domain logic that doesn't fit inside a single entity
  - Operate across multiple aggregates
  - Pure business logic, no infrastructure dependencies
  - Example: PricingService, InventoryChecker
- Application Services
  - Coordinate use cases and workflows
  - Call domain services and manage transactions
  - Define interfaces to be implemented by outer layers
  - Example: OrderService, CheckoutService
- Infrastructure (Outer Layer)
  - Implements interfaces defined in application/domain layers
  - Includes database access, external APIs, messaging, web frameworks
  - Can be replaced or mocked during testing
  - Example: SqlOrderRepository, EmailSender, WebController
- Dependency Direction
  - Inner layers define interfaces
  - Outer layers implement them
  - All dependencies point inward toward the domain core



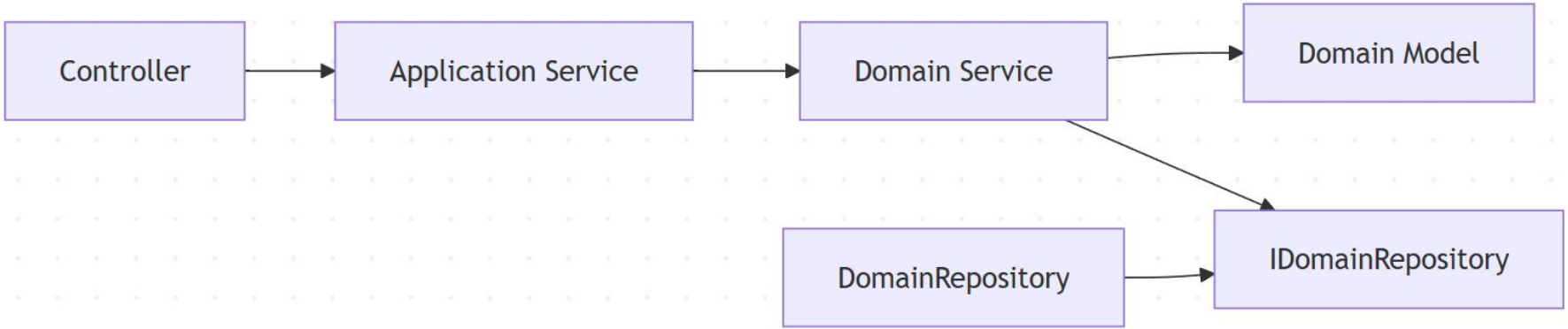
Dependency Inversion in Onion

- Domain defines interfaces
- Infra plugs into those abstractions
- No domain knowledge in outer layers



Onion Architecture in Action (Example Use Case)

- Use Case: Place Order
- Flow:
  - Controller calls App Service
  - App Service invokes domain logic
  - Domain logic calls interfaces
  - Infra implements them



Core Similarities - Onion and clean architecture

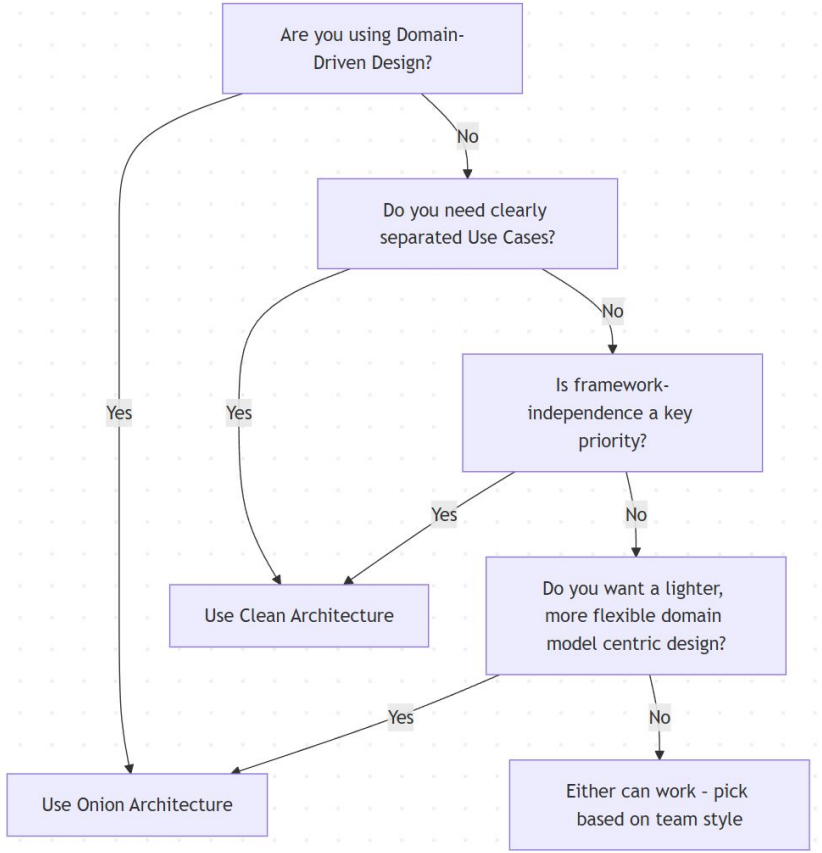
- Domain-centric core
- Inward dependencies
- Framework agnostic
- Easy unit testing
- Interfaces owned by core

Key Differences – Clean vs. Onion Architecture

- Terminology & Naming
  - Clean uses terms like Entities, Use Cases, Interface Adapters
  - Onion prefers Domain Model, Domain Services, Application Services
- Layer Granularity
  - Clean is more prescriptive with explicit layer boundaries
  - Onion is more flexible and often aligned with Domain-Driven Design (DDD)
- Flow Emphasis
  - Clean focuses heavily on application behavior (use cases)
  - Onion emphasizes the domain model and aggregates as the center
- Infrastructure Role
  - Clean clearly separates adapters and infrastructure (frameworks & drivers)
  - Onion includes infrastructure as the outermost ring implementing interfaces
- Adoption Style
  - Clean architecture often fits framework-agnostic application design
  - Onion architecture is typically adopted in DDD-heavy systems
- Diagram Style
  - Clean: Bullseye/target diagram with labeled concentric rings
  - Onion: Layered onion with the domain at the core and services around it

Choosing the Right Architecture

- Use Clean if:
  - Explicit use case handling needed
  - Framework-agnostic logic
  - You want clear separation of concern
- Use Onion if:
  - Domain-Driven Design (DDD) already used
  - You want domain model to lead
  - You’re building service-oriented apps



Common Mistakes to Avoid

- Domain layer calling infrastructure directly
- Leaky abstractions
- Putting logic in controllers
- Over-engineering for simple apps

Summary - Clean and Onion Architectures

- Clean & Onion share powerful core ideas
- They enable testability, maintainability, and flexibility
- Pick the approach that suits your team and context
- Keep your domain independent — always
- What’s next:
  - Applying Modular Architectures in Real Systems

Applying Modular Architectures in Real Systems

Layered & Modular Architecture Patterns

Mastering Software Architecture Patterns and System Design

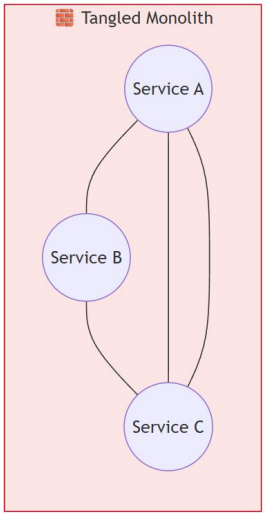
Recap – Where We Are

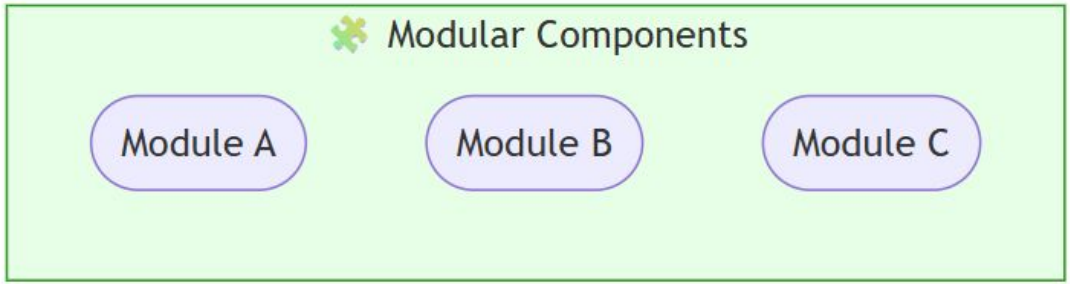
- Layered (3-tier/N-tier)
- Hexagonal (Ports & Adapters)
- Domain-centric Layering (Clean & Onion)
- ✅ Now: Applying these ideas modularly



Why Modular Architecture?

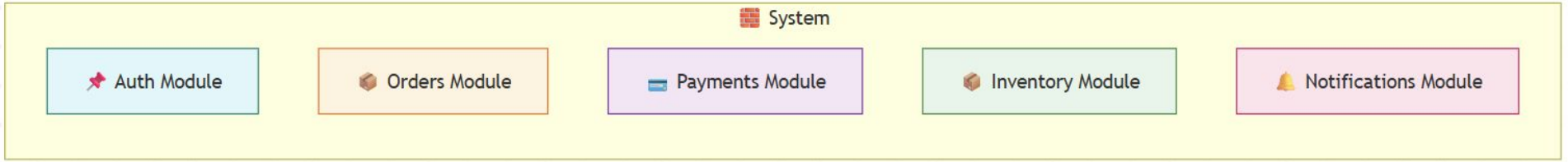
- Monoliths grow fast, but become hard to manage
- Helps large teams work independently
- Simplifies testing, deployment, and scaling
- Reduces coupling across the codebase





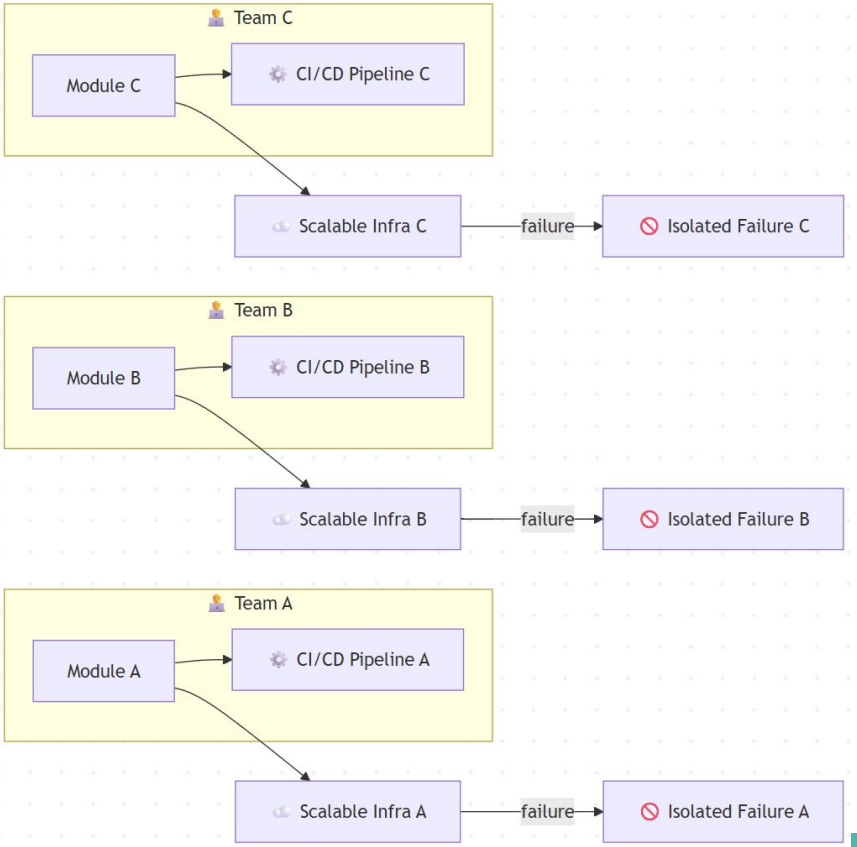
What Do We Mean by Module?

- A cohesive unit of functionality (like a mini-application)
- Has its own domain logic, services, interfaces
- Should be developable, testable, and deployable in isolation
- Not just a folder — it’s a boundary



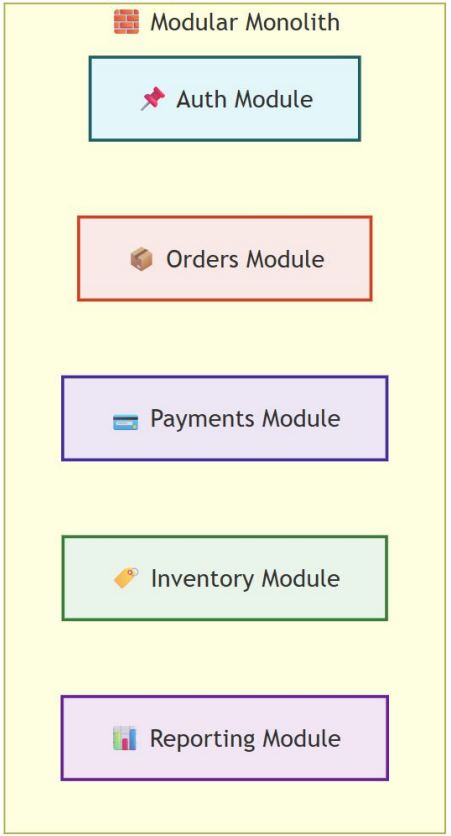
Benefits of Modular Architecture

- Independent team velocity
- Better CI/CD management
- Easier to scale horizontally
- Reduced blast radius of change



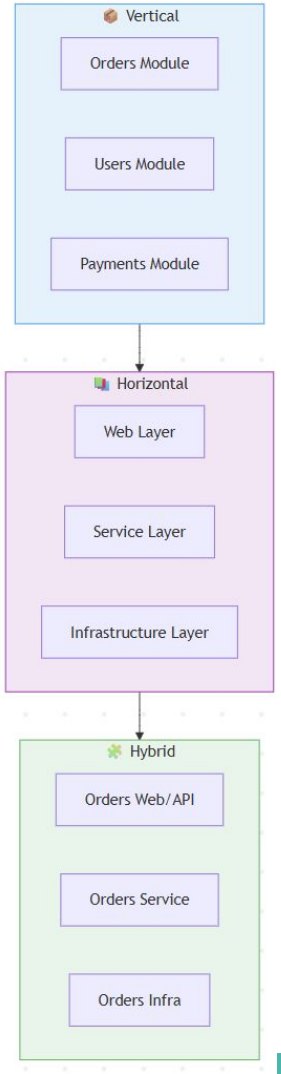
Modular Monolith – A Pragmatic Middle Ground

- Single deployable unit
- Internal structure mimics service boundaries
- Great stepping stone before microservices
- Avoids premature distributed complexity



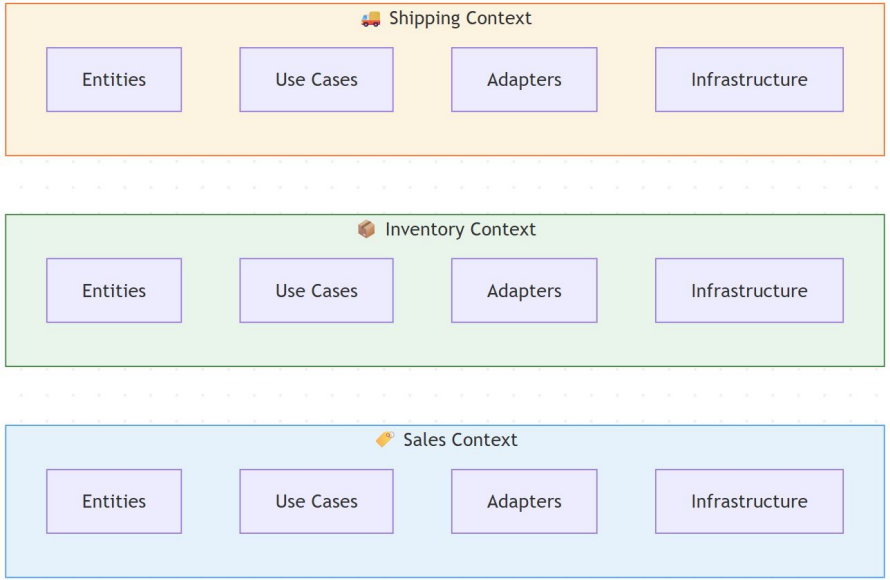
Styles of Modularity

- Vertical (Feature-Based): Orders, Users, Billing
- Horizontal (Layered): Core, Web, Infrastructure
- Hybrid (DDD-style Slices): Each module contains domain + service + adapter



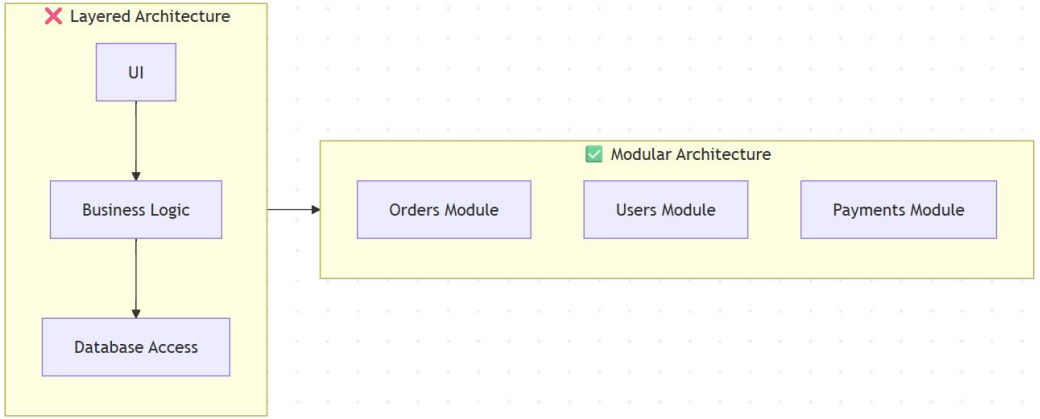
Modularity in Domain-Driven Design (DDD)

- Bounded Contexts make natural module boundaries
- Each module owns its own domain model and contracts
- Communicates via events or interface contracts
- Onion/Clean layering inside each module works well



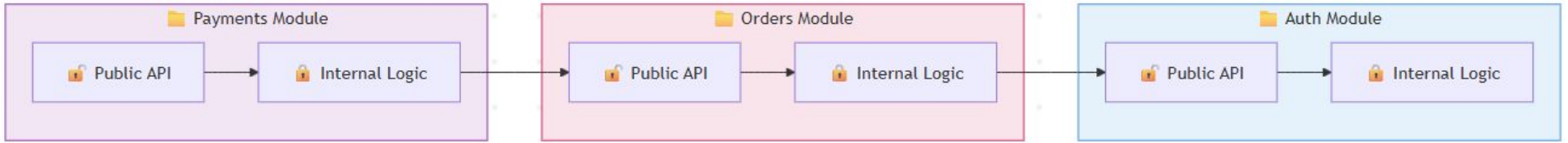
Evolving Layered Architectures into Modular

- Identify naturally grouped features in the monolith
- Move into folders/modules/projects
- Decouple slowly: extract interfaces, break shared dependencies
- Use anti-corruption layers when needed



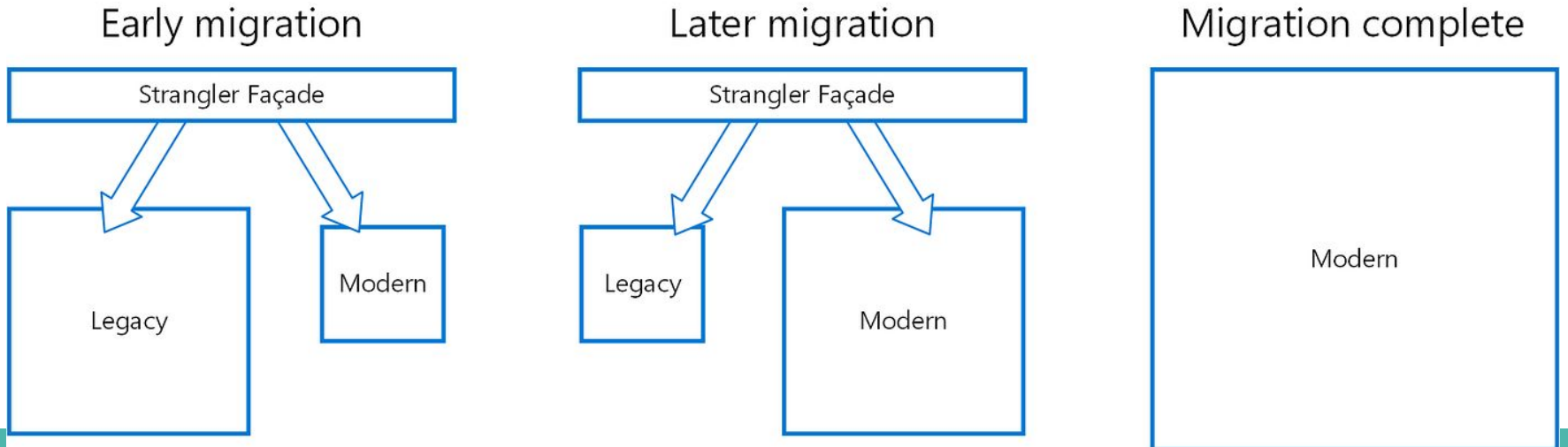
Technical Strategies for Enforcing Modularity

- Namespace/package segmentation
- Independent builds (e.g., Maven, Gradle, .NET projects)
- Visibility rules, linting, architectural boundaries
- Internal APIs vs shared/public APIs



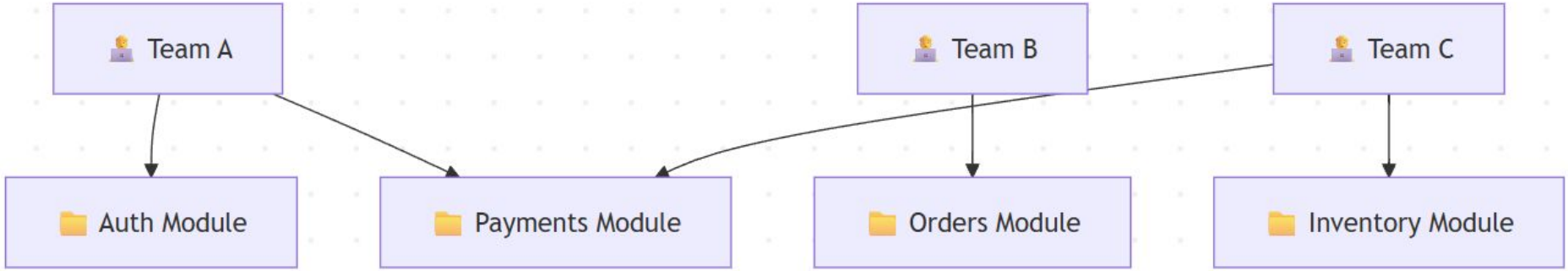
Migration Patterns from Legacy to Modular

- Strangler Pattern: replace feature-by-feature
- Façade Layer: sit in front of legacy and route calls
- Sidecar modules: add read-only new features first
- Define contracts before moving logic



Organizational Alignment for Modularity

- Map teams to modules or bounded contexts
- Each team owns build + release of their module
- Avoid shared ownership of critical modules
- Match code architecture with team topology (Conway’s Law!)



Summary – Modular Architectures

- Modularization = scaling code and teams
- Not all systems need microservices — modular monoliths are a valid goal
- Use DDD and layering together inside modules
- Boundaries buy long-term flexibility
- Treat modularity as both a design and organizational decision
- What’s next:
  - Distributed Systems & Service Architectures